When Does Waiting Help? Observable-History Retry Control for API-Dependent Agentic AI Workflows

Yukai Zeng*
Singapore Ministry of Defence Communications Organisation (MCO), Singapore
Corresponding author: Zeng_Yukai@defence.gov.sg

***Abstract*—Agentic AI workflows call networked search, retrieval and checking services, and when a call fails the client cannot tell an independent fault from a shared outage. A simulator can, and letting a policy see that distinction distorts how it is judged: a policy that read our simulator's hidden outage label delivered 92.0% of briefs on time and fresh, but only 24.5% once outages were masked as ordinary 5xx responses. We therefore test a three-parameter controller that picks a short or a long wait from status codes, `Retry-After` hints and its last 6 s of responses. Tuned on development data and frozen, it was compared with four baselines in 15,000 held-out trials on three strategic-communications workflows, 3,600 trials on unseen graphs and 320 localhost HTTP runs. It stayed within 1.5 percentage points of exponential backoff everywhere and beat it by 21–38 points under a masked extended outage. It tied a conservative linear cooldown on average (-0.3 points, 95% CI -1.7 to $+1.2$), losing only under that outage. A higher threshold cut its false alarms from 26% to 8% but its outage detection from 64% to 37%. When retries could prolong outages, the controller kept its lead over backoff, but the extra load favoured the cooldown.**

***Index Terms*—agentic AI workflows, retry control, fault tolerance, HTTP 429 and 503, circuit breaker, quality of service**

I. Introduction

An AI agent that searches for, retrieves and cross-checks sources makes many calls to networked tools, and some of those calls fail. Retrying improves the chance of finishing, but each retry spends time and request capacity. In time-sensitive work such as information-environment assessment, an AI support capability identified by the NATO StratCom Centre of Excellence [1], a slow retrieval can also leave material out of date by the time a brief reaches the analyst who reviews it.

Agent benchmarks such as AgentBench and τ-bench measure multi-step tool use [2], [3], and newer ones inject tool faults such as timeouts and rate limits [4], [5]. Those studies evaluate the agent; we study how its tool client should respond when a call fails. Adapting retries to observed history is an old idea: TCP sets its retransmission timeout from smoothed round-trip measurements [6]. Retries add tail latency and load [7], [8] and can keep an overloaded service from recovering [9], [10], so production clients cap them with budgets [11], throttle them when failures accumulate [12], [13], or stop calling a failing service with a circuit breaker [14], [15]. All of these act only on what the client can observe.

TABLE I
Label-masking control: timely fresh delivery (%, 200 seeds) when the outage cause is visible to the policy (Vis.) or reported as a generic 5xx (Mask.). 45-s deadline, 8% background errors, unextended outages.

	IEA		LFC		CSR	
Policy	Vis.	Mask.	Vis.	Mask.	Vis.	Mask.
Immediate	25.5	25.5	13.0	13.0	27.0	27.0
Exponential backoff	70.5	70.5	68.0	68.0	72.0	72.0
Cause-aware	92.0	24.5	83.0	12.5	92.0	26.5
Linear cooldown	93.5	93.5	72.0	72.0	91.5	91.5

A retry policy can look strong in simulation simply because it reads the simulator's outage label, which a real client never receives. Table I shows the size of the effect. The cause-aware policy in the table waits $1.5a$ s after a failure labelled `outage` and $0.2a$ s after any other 5xx or timeout, where a is the attempt number; it also caps calls per workflow and skips retries that a critical-path estimate says cannot finish in time. With the label visible it delivered 92.0% of information-environment briefs on time and fresh; with the same outages reported as ordinary 5xx responses it delivered 24.5%, and the other two workflows fell by similar margins. Immediate retry, exponential backoff and linear cooldown never read the label, and their outcomes were identical seed for seed in both conditions. The label-blind cooldown, which waits long after every 5xx, matched the cause-aware policy in IEA and CSR but pays for this under independent faults (Section V-A). The rest of the paper asks whether a client that sees only its own recent responses can wait long only when it should.

The paper makes three contributions. First, the label-masking control shows how easily a retry evaluation can hand a policy information that a real client lacks. Second, a three-parameter controller, tuned on development data and frozen, is compared with four baselines on held-out workflows, unseen graphs and real localhost HTTP requests. We do not claim it beats every fixed policy (it ties the linear cooldown on average); we use it to measure how much of the benefit of waiting a client can recover from its own response history. Third, post-hoc diagnostics show where that stops: sweeps over outage length, history window and threshold, outages that retries prolong, and an ablation against perfect detection. No experiment involves language-model inference, fact verifica-

TABLE II
RETRY DELAY SCHEDULES. a: ATTEMPT NUMBER; $j \in [0.85, 1.15]$: SEEDED JITTER; n: TASK COUNT. SUMS ARE OVER THREE RETRIES, BEFORE JITTER.

Policy	Delay before next attempt (s)	Σ (s)
Immediate	0 (ignores `Retry-After`)	0
Exponential backoff	$\min(8,\ 0.8 \cdot 2^{a-1})\ j$	5.6
SDK-style proxy	exponential, plus call cap $\lfloor 1.8n + 0.5 \rfloor$	5.6
Linear cooldown	$\min(5,\ 1.5a)\ j$; timeouts $0.2a\ j$	9.0
History-adaptive	$\min(5,\ \beta a j)$, $\beta \in \{0.4, 2.0\}$ by Eq. (2)	2.4 / 11.0

tion or production traffic.

II. RETRY POLICIES COMPARED

HTTP 429 tells a client that it is being throttled, and a `Retry-After` header can say how long to wait [16], [17]. AWS SDKs pair jittered backoff with a retry quota [13], gRPC suspends retries to a server once too many recent calls to it have failed [12], and circuit breakers open when the failure rate over a sliding window passes a threshold [15]. Our controller belongs to this family and adds no new detection mechanism; the contribution is a comparison in which every policy sees the same information and is scored on whether a dependent workflow delivers fresh results on time.

Table II lists the five policies. After a 429, every policy except Immediate waits at least as long as `Retry-After` asks. Permanent errors are never retried, and no task gets more than four attempts. With that limit, the longest total wait in the table (11 s) still fits inside the shortest held-out deadline (14 s). All policies see timestamps, task dependencies, status codes and `Retry-After` values. None sees the true service state or any future draw.

The SDK-style proxy is exponential backoff plus a per-workflow call cap; it does not run an AWS SDK. It keeps the two mechanisms that can act within one workflow and omits state that production libraries keep per client. In the standard mode as revised by AWS in 2026, a retry after a non-throttling error takes 14 tokens from a 500-token per-client bucket [13]; a reporting workflow here makes at most 30 retries (420 tokens), so the quota cannot run out, and the mode's 50-ms base delay would put it close to Immediate after a 5xx. A Resilience4j circuit breaker with default settings needs 100 calls before it computes a failure rate [15]; our workflows make at most 72.

III. WORKFLOW SIMULATOR AND CONTROLLER

A. Synthetic reporting workflows and metric

We use three fixed directed acyclic graph (DAG) templates: information-environment assessment (IEA, 10 tasks), live fact-check support (LFC, 8) and communications situation reporting (CSR, 9). Each follows the generic stages of an assessment brief: public-source retrieval, mandatory provenance and consistency checks, optional enrichment, and handoff to a human-review queue. A completed check means only that the task ran, and the templates do not reproduce any organisation's procedure.

Let D_s^π indicate brief delivery for seed s and policy π, M_s^π completion of the required checks, Δ the deadline, and R the set of mandatory retrievals, each with a freshness TTL τ_i (20–35 simulated seconds, chosen for illustration and not calibrated). With t_s^π the delivery time and $t_{i,s}^\pi$ the completion time of retrieval i, the endpoint is timely fresh delivery

$$Y_s^\pi = \mathbf{1}\left[D_s^\pi \wedge M_s^\pi \wedge t_s^\pi \le \Delta \wedge \bigwedge_{i \in R} (t_s^\pi - t_{i,s}^\pi \le \tau_i) \right]. \quad (1)$$

For policies A, B, the paired effect is $\widehat{\delta}_{A,B} = N^{-1} \sum_s (Y_s^A - Y_s^B)$, with 95% percentile bootstrap intervals that resample seeds [18].

For each seed, every task has fixed error and latency draws for each attempt, with latencies from a bounded log-normal distribution (median 1.25 s). Four workers execute the DAG as a discrete-event simulation. Each seed also has two shared outage windows: one of 6.5 s starting between 4.5 and 7.5 s, and one of 3.0 s starting between 19 and 23 s. Any request sent during a window fails fast (0.2 s) with a 5xx. All policies share the same draws, but outages and the 429 token bucket depend on when a request is sent, so two policies can meet different failures on the same seed.

B. Observable-history retry controller

At time t, let H_t hold the *completed* responses (u, z) from the preceding $W = 6$ s of the same workflow, where u is the completion time and z the outcome. We set W before tuning to just under the 6.5-s first outage window. That is a simulator-informed choice, which a real client could not make; Section V-D tests 2 and 12 s and discusses calibration. Let $e_t = \sum_{(u,z) \in H_t} \mathbf{1}[z \in \{5\text{xx}, \text{timeout}\}]$ count the recent 5xx responses and timeouts. After a retryable 5xx or timeout at attempt a, the controller waits

$$d(t, a) = \min(5, a\, j\, [\beta_{\text{long}}\, \mathbf{1}(e_t \ge k) + \beta_{\text{short}}\, \mathbf{1}(e_t < k)])\ \text{s}. \quad (2)$$

After a 429 it waits $\max(0.8a, r)$ s, where r is the `Retry-After` value. It also skips any retry that would leave less than 0.1 s before the deadline; this guard changed no outcome in the 3,000 held-out history-adaptive trials.

IV. EVALUATION PROTOCOL

A. Development tuning, then freeze

A development grid crossed $k \in \{2, 3\}$, $\beta_{\text{short}} \in \{0.2, 0.4\}$ and $\beta_{\text{long}} \in \{1, 1.5, 2\}$ with $W = 6$ s fixed: 12 configurations $\times$ 360 trials (three templates, three development conditions, 40 seeds) $= 4{,}320$ records. The development conditions were 22% IID errors with a 22-s deadline, an unextended outage with 7% background errors, and a hinted 1.8 requests/s quota. Selection used pooled timely fresh delivery, with mean calls as a tie-break, and chose $k = 3$, $\beta_{\text{short}} = 0.4$, $\beta_{\text{long}} = 2.0$ (96.1% on development data). We froze that setting in a JSON file before running any held-out trial. A post-hoc 120-configuration grid on the same development seeds ($k \le 5$, $\beta_{\text{short}} \le 1.5$, $\beta_{\text{long}} \le 3$) still ranked it first.

B. Held-out and structural tests

Five policies were run on three templates and five held-out conditions, each with 200 new seeds ($5 \times 3 \times 5 \times 200 = 15{,}000$ records): independent and identically distributed (IID) errors at 27%, a masked outage with both windows extended by 2 s, tight IID errors (17%, 14-s deadline), a hinted quota (2.3 requests/s) and mixed permanent/transient errors. The tight-deadline and mixed conditions have no counterpart in development. A further 3,600 records (three topologies $\times$ two failure conditions $\times$ five policies $\times$ 120 seeds) use unseen 8-node chains, 15-node fan-outs and 18-node layered DAGs. These graphs are scored by *generic completion* rather than freshness. Seed ranges for development, held-out, structural and HTTP runs do not overlap.

C. Localhost HTTP integration test

A standard-library server and client exchanged real requests over `127.0.0.1`, with injected 503s, 429s carrying `Retry-After`, timeouts and a brief outage. Each of the 320 records (four regimes $\times$ five policies $\times$ 16 seeds) is a workflow of three sequential calls, with time accelerated to 35 ms per nominal second.

D. Post-hoc diagnostics

Further analyses used the held-out seeds after the primary test and never fed back into the configuration: sweeps over outage length (0–6 s of extension, 18,000 records, whose +2-s slice matches the primary records exactly), history window ($W \in \{2, 6, 12\}$ s) and threshold; a detector check that uses the hidden outage label *only to score* the controller's state; a switching ablation (18,000 records) that fixes the controller's short or long wait or lets the hidden label choose (perfect detection, an upper bound); and outages that retries prolong.

V. Results

A. Held-out rankings

Table III gives all fifteen held-out cells. History-adaptive retry was never more than 1.5 points behind exponential backoff (LFC, 27% IID), the usual SDK default [12], [13], and was far ahead of it under the extended outage, by 21.0 to 37.5 points across the three workflows. The linear cooldown recovered more of the extended outage in every workflow (74.0% against 60.5% in IEA), and immediate retry delivered more in every independent-fault condition, most of all under tight deadlines (95.0% against 80.5% in IEA). Because immediate retry ignores `Retry-After`, its results under the hinted quota swing with the workflow, from 99.5% in LFC to 0.0% in CSR. The SDK-style proxy's call cap bound in only 12 of the 3,000 held-out trials and changed four outcomes; otherwise the proxy matched exponential backoff.

Averaged over the fifteen cells, history-adaptive retry fell 8.0 points short of each cell's best policy, against 7.7 for the cooldown, 14.3 for exponential backoff and 20.4 for immediate retry. Pooled over the cells, the controller was 0.3 points behind the cooldown (95% CI -1.7 to $+1.2$): a tie, with failures in opposite places. The controller led in all twelve

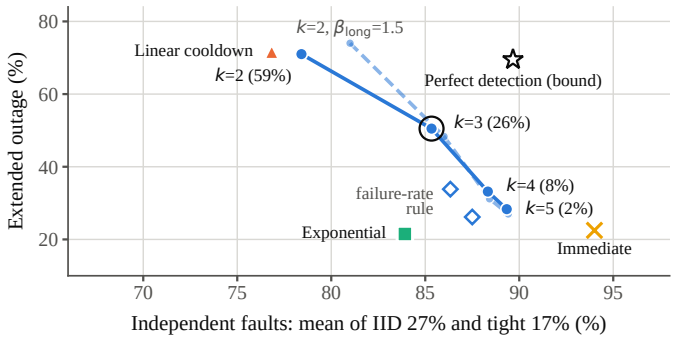


Fig. 1. Post-hoc threshold sweep, pooled over workflows (600 runs per condition). Solid: $\beta_{\text{long}} = 2$, $k = 2$–5 (frozen circled); dashed: $\beta_{\text{long}} = 1.5$; diamonds: failure-rate rule, $k = 3$. Brackets: long waits after independent errors at 27% IID. Perfect detection reads the hidden label.

cells outside the extended outage, by 5.0 points on average, and its worst shortfall from a cell's best policy (26.5 points, LFC outage) was close to the cooldown's (26.0, LFC tight deadlines). The tie also reflects the frozen choice: the two development settings ranked next ($k = 2$, $\beta_{\text{long}} = 1.5$), 0.8 points lower on development data (three of 360 trials), would have matched or beaten the cooldown in 14 and 15 of the fifteen held-out cells. We note this post hoc and keep the frozen setting.

B. Delay budgets and outage detection

With at most four attempts per task, a policy gets through an outage only if its waits add up to more than the outage time remaining when its first failure arrives. The held-out windows last 8.5 s and 5.0 s. Exponential backoff waits 5.6 s in total (Table II), which is usually too little for the longer window, while the cooldown's 9.0 s usually covers it. History-adaptive retry waits either 2.4 s or 11.0 s, so its result hinges on noticing the outage. Scored against the hidden label, it chose the long wait for 64% of the 1,808 retries that followed outage-caused failures. It also chose the long wait after independent errors, for 26% of those retries at the 27% error rate and 14% at the 17% rate.

The switching ablation separates the cost of these errors from the cost of the waits. Used alone, the short wait collapsed under the extended outage (14.5–23.5%), and the long wait lost 16 to 23 points to the controller under tight deadlines; the controller lay between the two in every held-out cell. With perfect detection, the same waits drew level with the cooldown under the extended outage in IEA and CSR (+1.0 points, intervals spanning zero) and recovered about two-thirds of the gap in LFC, so detection errors explain most of the outage loss. Under tight deadlines perfect detection only reached the short wait's result, still 7 to 9 points behind immediate retry. Pooled over the workflows, false alarms cost 5.2 points there and 3.5 at 27% errors; they explain less than half of the gap to immediate retry, and the rest is the cost of waiting at all.

C. Can false alarms be tuned away?

Fig. 1 reruns the controller with $k = 2$–5 and $\beta_{\text{long}} = 1$–3, keeping β_{short} and W. The false-alarm rate is set by k:

TABLE III

HELD-OUT TIMELY FRESH DELIVERY (%, $N = 200$ MATCHED SEEDS PER CELL). DEADLINES AND BACKGROUND ERRORS: IID, 26 S; EXTENDED OUTAGE, 48 S, 10% IID; TIGHT, 14 S; HINTED QUOTA (2.3 REQUESTS/S), 52 S, 9% IID; MIXED, 40 S, 20% IID AND 2% PERMANENT. IMM.: IMMEDIATE; EXP.: EXPONENTIAL BACKOFF; SDK: SDK-STYLE PROXY; COOL.: LINEAR COOLDOWN; HIST.: HISTORY-ADAPTIVE. BOLD: BEST IN THE ROW WITHIN EACH WORKFLOW, TIES INCLUDED.

	IEA					LFC					CSR				
Condition	Imm.	Exp.	SDK	Cool.	Hist.	Imm.	Exp.	SDK	Cool.	Hist.	Imm.	Exp.	SDK	Cool.	Hist.
IID 27%	**95.0**	92.5	92.5	87.0	93.0	**95.5**	89.0	89.0	81.0	87.5	**94.0**	92.0	91.0	87.5	91.5
Extended outage	26.0	23.0	23.0	**74.0**	60.5	16.0	18.5	18.5	**66.0**	39.5	25.5	23.0	23.0	**75.0**	51.5
Tight IID 17%	**95.0**	78.5	78.5	71.5	80.5	**91.0**	76.0	76.0	65.0	81.0	**93.5**	75.5	75.5	69.0	78.5
Hinted quota	41.5	**99.5**	**99.5**	98.0	99.0	**99.5**	**99.5**	**99.5**	98.5	**99.5**	0.0	**100.0**	**100.0**	99.0	**100.0**
Mixed permanent	**83.5**	82.5	82.5	81.0	82.5	**84.5**	82.0	81.5	79.5	82.0	**82.0**	81.5	81.0	80.0	81.5

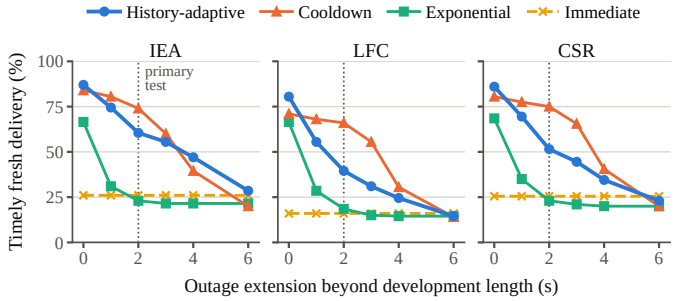


Fig. 2. Post-hoc outage-length sweep (200 held-out seeds per point). The +2-s point is the primary test. The SDK-style proxy overlaps exponential backoff and is omitted.

at 27% errors it fell from 59% of independent-error retries at $k = 2$ to 26%, 8% and 2.5% at $k = 3$, 4 and 5, but detection fell from 88% to 64%, 37% and 25% of outage-driven retries, and outage delivery from 71.0% to 50.5%, 33.2% and 28.3%. Changing β_{long} hardly altered how often the long wait was chosen (24–29% at $k = 3$); it set what each false alarm cost and whether the long waits outlasted the outage (at $\beta_{\text{long}} = 1$ they did not, and outage delivery fell to 21.7%). A circuit-breaker-style rule that also required half of the recent responses to have failed gave 17% false alarms and 35% detection, no better than $k = 4$. The settings ran from near immediate retry ($k = 5$) to near the cooldown ($k = 2$), and none came within 5 points of perfect detection on both axes: fewer false alarms need a better detector, not a different threshold.

D. History-window sensitivity

A longer history window helps under outages and hurts under independent errors: in IEA, moving from $W = 2$ to 12 s lowered delivery under independent errors from 95.0% to 87.0% and raised it under the extended outage from 40.0% to 63.0%, and LFC and CSR moved the same way. A deployed client would calibrate W and k together from its own logs, not from known outage windows. Replaying logged responses through the controller gives its false-alarm rate at the observed background error rate, incident records give outage lengths that W should not exceed, and the pair can be chosen to meet a false-alarm budget while still detecting the logged outages.

TABLE IV

LOAD-COUPLED OUTAGES (POST HOC): TIMELY FRESH DELIVERY (%), POOLED OVER WORKFLOWS (600 RUNS PER CELL). EACH REQUEST RECEIVED DURING AN OUTAGE DELAYS ITS END BY δ S. IMMEDIATE RETRY: 22.5% THROUGHOUT; SDK-STYLE PROXY WITHIN 1.5 POINTS OF EXP. BOLD: BEST PER ROW AND OUTAGE LENGTH.

	Development length			Extended (+2 s)		
δ (s)	Exp.	Cool.	Hist.	Exp.	Cool.	Hist.
0	67.2	78.5	**84.5**	21.5	**71.7**	50.5
0.1	50.7	77.7	**82.7**	20.5	**68.8**	48.0
0.2	40.8	76.0	**76.3**	19.5	**62.5**	45.5
0.3	32.5	**75.2**	69.8	19.5	**56.5**	41.5
0.5	26.0	**69.3**	54.3	18.7	**42.5**	33.5
1	20.0	**54.5**	35.2	18.7	22.0	**22.2**

E. Outage-length sensitivity

Fig. 2 lengthens both outage windows by 0–6 s. The primary +2-s test is where the cooldown's lead is largest. At the development length, history-adaptive retry led the cooldown in all three workflows (by 3.0–9.5 points; intervals excluding zero in LFC and CSR). With 1–3 s of extension the cooldown led (intervals excluding zero at all three lengths in LFC and CSR, in IEA only at +2 s). At +4 and +6 s history-adaptive retry led again in IEA. Exponential backoff fell from 66.5% to 31.0% in IEA with one extra second, and history-adaptive retry stayed at or above it at every length.

F. Outages that retries prolong

In the primary study outages are scheduled in advance, so retries cannot prolong them; in Table IV every request that reaches the service during an outage delays its end by δ s (δ can also stand for other clients' retries). A workflow sent only 2.8–4.0 requests into an outage on average, because tasks queued behind a failed retrieval send nothing, so load mainly lengthened the outage: at $\delta = 0.3$ s (about 1 s more) the development-length results (69.8% history-adaptive, 75.2% cooldown, 32.5% backoff) were close to the +1-s point of Fig. 2 (66.5%, 75.3%, 31.5%).

The independent-fault and quota results have no outage windows and cannot depend on the assumption; the quota condition already has load feedback through its token bucket, under which the policies that honour `Retry-After` stayed at 98.0–100%. The label-masking gap held: with the label visible, the cause-aware policy stayed at least 34 points ahead

of its masked runs at every δ. So did the controller's gain over exponential backoff, at every δ and both lengths (smallest +3.5 points, CI 2.2–5.0). The comparison with the cooldown does depend on it: at the development length the controller's 6.0-point lead (CI 3.3–8.8) vanished at $\delta = 0.2$ and reversed at $\delta \geq 0.3$ (-5.3, CI -9.0 to -1.5), while at the extended length the cooldown's 21.2-point lead narrowed only as every policy fell towards immediate retry's 22.5%. If retries prolong outages, the case for the controller over the cooldown is weaker than Table III suggests.

G. Unseen graphs and localhost HTTP

On unseen chain, fan-out and layered graphs (120 seeds each), history-adaptive retry completed 76–91% of workflows under outages, against 35–68% for exponential backoff and 0–47% for immediate retry, and was within one point of the cooldown on every graph. Under independent errors the policies were indistinguishable.

Over localhost HTTP, every waiting policy completed all 16 runs of the brief outage and of the shared 429 limit, and immediate retry none; under IID 503s and timeouts the policies differed by at most one run. The test establishes that the simulation's decision code runs unchanged against a real socket server: it reads status codes and `Retry-After` headers, treats client timeouts as retryable and keeps its waits in wall-clock time. It cannot rank the waiting policies, which all passed every shared-fault run on one fixed schedule per regime, and it says nothing about real providers, network latency or concurrent clients.

VI. Limitations and Reproducibility

The DAGs, TTLs, deadlines and outage windows are illustrative rather than calibrated. The intervals describe seed-to-seed variation within our generator, and the exploratory comparisons are not corrected for multiplicity. Load feedback is only partly modelled: retries can prolong an outage (Section V-F), but outages still begin on a schedule, and with one workflow and no queueing the retry storms and metastable overload of [9], [10] cannot arise. The controller's history is also reset for each workflow. A comparison with production SDK and circuit-breaker libraries would add the state they share across a client's calls: retry quotas, adaptive rate limiters and breakers, which see failures from all workflows at once and, when open, stop calls and relieve the failing service. Testing them needs many concurrent workflows behind one client and a load-dependent service.

All simulated tasks are idempotent; real tools that publish, submit or update records would need idempotency keys or operator review before a retry. Freshness is measured by time alone, so $Y = 1$ says nothing about whether a brief's content was right or approved. The code at https://github.com/yukai83/RetryBench-StratCom contains the simulator, the frozen configuration, per-seed CSVs, audit scripts, a checker for every number quoted here and 98 automated tests.

VII. Conclusion

Letting a retry policy read the simulator's outage label made its outage performance look almost four times better than it was, and the gap persisted when retries prolonged outages. Without the label, a three-parameter controller stayed within 1.5 points of exponential backoff in every held-out condition and beat it by 21–38 points under the masked extended outage, a lead that held under load feedback and on unseen graphs. It tied the linear cooldown on average, winning everywhere except under the extended outage, and load favoured the cooldown. No threshold escaped the trade-off between false alarms and detection. Next come better detection, a client-wide history, and tests against load-dependent services, real SDK and breaker libraries and permissioned traces.

AI-Use Disclosure

Claude (Anthropic) and ChatGPT (OpenAI) were used for language editing. The author reviewed all changes and takes responsibility for the content.

References

[1] G. Bergmanis-Korāts, G. Bertolin, A. Pužule, and Y. Zeng, *AI in Support of StratCom Capabilities*, M. Read, Ed. Riga, Latvia: NATO Strategic Communications Centre of Excellence, 2024, ISBN: 978-9934-619-55-7.

[2] X. Liu *et al.*, "AgentBench: Evaluating LLMs as agents," in *Proc. Int. Conf. Learn. Representations (ICLR)*, 2024.

[3] S. Yao, N. Shinn, P. Razavi, and K. Narasimhan, "τ-bench: A benchmark for tool-agent-user interaction in real-world domains," in *Proc. Int. Conf. Learn. Representations (ICLR)*, 2025.

[4] A. Gupta, "ReliabilityBench: Evaluating LLM agent reliability under production-like stress conditions," arXiv:2601.06112, 2026.

[5] Y. Tian, Z. Shi, Y. Zhou, and B. Zhao, "Beyond function calling: Benchmarking tool-using agents under tool-environment unreliability," arXiv:2606.25819, 2026.

[6] V. Paxson, M. Allman, J. Chu, and M. Sargent, "Computing TCP's retransmission timer," RFC 6298, 2011, doi: 10.17487/RFC6298.

[7] J. Dean and L. A. Barroso, "The tail at scale," *Commun. ACM*, vol. 56, no. 2, pp. 74–80, 2013, doi: 10.1145/2408776.2408794.

[8] M. Brooker, "Timeouts, retries, and backoff with jitter," *The Amazon Builders' Library*, Amazon Web Services, Dec. 2019. [Online]. Available: https://builder.aws.com/content/3EumjoZascWd1oZiEgL8ORlv3qE/timeouts-retries-and-backoff-with-jitter. Accessed: Oct. 23, 2026.

[9] N. Bronson, A. Aghayev, A. Charapko, and T. Zhu, "Metastable failures in distributed systems," in *Proc. Workshop Hot Topics Oper. Syst. (HotOS)*, 2021, pp. 221–227, doi: 10.1145/3458336.3465286.

[10] L. Huang *et al.*, "Metastable failures in the wild," in *Proc. 16th USENIX Symp. Oper. Syst. Des. Implement. (OSDI)*, 2022, pp. 73–90.

[11] B. Beyer, C. Jones, J. Petoff, and N. R. Murphy, Eds., *Site Reliability Engineering: How Google Runs Production Systems*. Sebastopol, CA, USA: O'Reilly Media, 2016, ch. 21.

[12] N. Eisen and E. Gribkoff, "gRPC retry design," gRPC proposal A6. [Online]. Available: https://github.com/grpc/proposal/blob/master/A6-client-retries.md. Accessed: Oct. 23, 2026.

[13] Amazon Web Services, "Retry behavior," *AWS SDKs and Tools Reference Guide*. [Online]. Available: https://docs.aws.amazon.com/sdkref/latest/guide/feature-retry-behavior.html. Accessed: Nov. 7, 2026.

[14] M. T. Nygard, *Release It! Design and Deploy Production-Ready Software*, 2nd ed. Raleigh, NC, USA: Pragmatic Bookshelf, 2018.

[15] Resilience4j, "CircuitBreaker," *Resilience4j Documentation*. [Online]. Available: https://resilience4j.readme.io/docs/circuitbreaker. Accessed: Oct. 23, 2026.

[16] M. Nottingham and R. Fielding, "Additional HTTP status codes," RFC 6585, 2012, doi: 10.17487/RFC6585.

[17] R. Fielding, M. Nottingham, and J. Reschke, "HTTP semantics," RFC 9110, 2022, doi: 10.17487/RFC9110.

[18] B. Efron, "Bootstrap methods: Another look at the jackknife," *Ann. Statist.*, vol. 7, no. 1, pp. 1–26, 1979, doi: 10.1214/aos/1176344552.